

CampusResolve

Project Submission Report

Student Support and Ticket Management

Edumerge Solutions | Assignment 4 | 24 September 2026

Candidate :Karthikeyan R

Source repository : https://github.com/KarthikeyanRamlingam/Student_ticket_Management_System.git

Prototype access : <https://campusresolve-jade.vercel.app>

Demo accounts : https://github.com/KarthikeyanRamlingam/Student_ticket_Management_System/blob/main/README.md

Project overview

CampusResolve centralizes student requests about fees, attendance, identity cards, documents, certificates, and other administrative matters. It gives students a way to submit and follow requests, staff a queue with clear ownership and next actions, and administrators visibility into workload, delays, and resolution performance.

The prototype combines a Next.js interface, an Express API, and PostgreSQL persistence. Its main engineering concerns are role-based access, valid ticket transitions, private internal notes, service-level agreement (SLA) tracking, and an activity history that explains how a request was handled. **Users and responsibilities**

Role	Main responsibilities
Student	Create and track personal tickets, add public replies, provide requested information, and accept a resolution.
Staff	Work on authorized tickets, claim ownership, update progress, request information, add internal notes, and resolve requests.
Admin	Oversee ticket queues, assignment, categories, workload, ageing, and SLA reporting.

Prototype scope

The implementation record covers authentication, ticket creation and claiming, status transitions, public and internal comments, priority-based SLA calculations, ageing, dashboards, and a seeded database with at least 22 tickets. The interface includes ticket lists, Grid and Kanban views, a ticket lifecycle stepper, a command palette, and toast notifications.

The report records AI contributions, developer modifications, defects found in generated output, and validation results. It also explains the product assumptions, engineering tradeoffs, and checks needed for the final submission.

Technical approach and workflow

I separated presentation, API handling, business rules, and persistence so that authorization and workflow decisions are enforced on the server and can be tested independently of the interface.

Layer	Technology and responsibility
Frontend	Next.js App Router, React, TypeScript, Tailwind CSS, and Recharts for pages, forms, and dashboard views.
Backend	Node.js and Express with TypeScript for APIs, middleware, validation, and business services.
Data	PostgreSQL and Prisma for relational data, schema changes, queries, and seed data.
Access and validation	JWT verification, role-based authorization, and Zod request validation.
Verification	Jest tests, TypeScript checking, a production frontend build, a lifecycle script, and automated browser checks.

Request handling and data design

Frontend requests reach Express routes and middleware before service methods read or update data through Prisma. Core records include users, departments, categories, tickets, comments, activities, and attachments. Tickets link the requesting student, category, department, and assigned staff member. Public ticket numbers such as TKT-2026-00022 provide a readable reference.

Comments carry a visibility value so public conversation and internal notes can share a ticket without sharing the same audience. Activity records capture changes to ownership and progress. Dashboard totals and chart series are calculated from stored ticket data.

Ticket lifecycle and pending actions

The demonstrated lifecycle starts with ticket creation and staff claiming, followed by active work. Staff can request more information by moving a ticket to `WAITING_FOR_STUDENT`. A public reply from the requesting student automatically returns that ticket to `IN_PROGRESS`. Staff then resolve the request, and the student can accept the resolution and close it.

The service layer uses an explicit transition map to reject invalid status changes. The pending-action flow makes the next responsible person visible: staff while work is underway, and the student while information is requested. The full transition map remains part of the source code.

SLA and management visibility

SLA calculations compare the current time or resolution timestamp with the ticket deadline. The UI distinguishes `WITHIN_SLA`, `DUE_SOON`, `OVERDUE`, `MET`, and `BREACHED`. Unresolved ticket age is grouped into 0-1, 2-3, 4-7, and 8+ day buckets. Dashboards show category and status distributions, ageing, and staff workload.

This makes delays visible without storing a permanently fixed overdue flag. Exact SLA durations, the due-soon threshold, and any pause or reopening rules should be documented alongside the configured policy.

Mandatory AI usage report

I used AI during requirements planning, architecture, implementation, debugging, test creation, and documentation. Generated code was part of the development process; it was reviewed and checked through the commands and workflows described in this report.

AI tools used

Tool	Use disclosed
Google Antigravity with Gemini	Primary AI-assisted development environment, used on Windows 11.
ChatGPT	Requirements discussion, the master build prompt, and preparation of this submission report.
Claude Sonnet	Additional AI assistance during development.

What I asked AI to do

1. Design and scaffold the separated frontend and backend, the PostgreSQL and Prisma schema, relationships, indexes, and realistic demo data.
2. Implement ticket ownership, role checks, status transitions, public and internal communication, SLA calculations, ageing, and dashboard queries.
3. Generate tests, diagnose build and environment errors, and create responsive pages, reusable components, and reporting charts.

Most useful prompt

“Build a centralized ticket management platform with Role-Based Access Control (Student, Staff, Admin), dynamic SLA calculations based on priority, atomic concurrency for ticket claiming, strict privacy where internal staff notes are never exposed to students, and a pending-action workflow where a student reply to a `WAITING_FOR_STUDENT` ticket automatically resumes it to `IN_PROGRESS`.”

This prompt was useful because it described the actors, privacy boundaries, ownership rules, and expected workflow. It gave the implementation and validation work concrete behaviors to check.

How AI output was evaluated

I used schema and service review to check the design, automated tests for business rules, build commands for compilation and prerendering, a scripted ticket lifecycle for API behavior, and automated browser checks for the interface. A passing build was treated as compilation evidence; privacy, workflow correctness, and concurrency require separate checks.

Generated code and developer review

Code generated with AI assistance

Area	Files or responsibilities
Infrastructure	Express configuration, CORS, errorHandler.ts, auth.ts, JWT verification, role checks, and Zod validators.
Database	schema.prisma relations and enums; seed.ts with tickets across departments, states, priorities, and SLA conditions.
Ticket services	ticket.service.ts for creation, numbering, claiming, and transitions; comment.service.ts for visibility and student replies.
SLA and analytics	slaCalculator.ts for deadlines and ageing; analytics.service.ts for dashboard aggregations from stored data.
Frontend	Login, registration, dashboard, tickets, ticket details, and categories pages; badges, navigation, ticket tables, forms, and charts.
Tests	Jest test scaffolding and assertions for SLA behavior, transitions, authorization, and internal comment privacy.

Code I personally reviewed and modified

Prisma relations and indexes. I reviewed cascade behavior for TicketComment, TicketActivity, and Attachment during test teardown. I added or reviewed single-column indexes on status, priority, studentId, assignedStaffId, and slaDueAt to support common filters. Their performance benefit would need query-plan or timing evidence before making a measured performance claim.

Claiming and conflict handling. I revised the claiming service to use a Prisma interactive transaction, check existing ownership, and return HTTP 409 when a claim conflicts. A transaction containing a read followed by an unconditional update does not, by itself, establish safe simultaneous claiming. The final check must confirm a conditional write, version check, or suitable locking or isolation strategy. [1]

Privacy at query level. I changed student-facing ticket and comment retrieval to filter visibility to PUBLIC in the Prisma query. This keeps internal note content out of student API responses. Related activity metadata and any other student-visible endpoints also need to preserve this boundary.

Frontend prerendering. I moved the useSearchParams consumer into a TicketsContent child component and wrapped that component in Suspense with a loading fallback. This addressed the reported production build failure on the tickets page. [2]

AI assistance remained part of the workflow, including code generation, debugging, and preparation of this report. My review focused on the schema, authorization paths, ticket behavior, and build results.

AI errors and validation results

Incorrect output and how I corrected it

Missing Suspense boundary. Generated client-side code used useSearchParams without the required surrounding boundary for static prerendering. I identified the problem when npm run build failed on /tickets. I extracted TicketsContent and rendered it inside Suspense with a Loader2 fallback. The subsequent frontend build was recorded as successful. [2]

Shell syntax mismatch. AI suggested Bash process-substitution syntax for PostgreSQL initialization in a Windows PowerShell session. A parser failure exposed the mismatch. I removed the Bash-specific password input and used PostgreSQL trust authentication for local bootstrap. That was a local workaround, not a secure deployment configuration; shared or hosted use requires appropriately restricted, authenticated database access. [3]

Recorded validation results

Check	Method	Recorded result
Automated tests	npm test with Jest	11 tests passed across SLA calculations, transitions, role authorization, and comment privacy.
Backend type check	npx tsc --noEmit	Exit code 0.
Frontend build	npm run build	9 routes built; 0 errors and 0 warnings reported.
API lifecycle	Automated scenario script	Ticket creation through student acceptance and closure completed.
Interface	AI-assisted browser validation	Charts, board grouping, command palette, lifecycle stepper, and toasts checked.

Ticket lifecycle checked

The scenario used ticket TKT-2026-00022. A student logged in and submitted the ticket. A staff user logged in, claimed it, and created an internal note; student retrieval was checked for note privacy. Staff requested more information, producing `WAITING_FOR_STUDENT`. The student replied and the ticket returned to `IN_PROGRESS`. Staff resolved it with SLA status `MET`, and the student accepted and closed it.

Interface checks and scope

Automated browser checks covered Recharts bar and donut charts, category distributions, Grid and Kanban grouping, Ctrl+K search and demo persona switching, the five-stage lifecycle display, and toast feedback. Persona switching is a demo convenience; it must not replace server-side authorization.

The results above are the developer validation record. They do not establish exhaustive test coverage, deployment readiness, load capacity, or safety under simultaneous claims. Preserve the command output and lifecycle script with the submitted source so the results can be reproduced.

Assumptions and submission details

Product assumptions

- The prototype serves one institution. Categories route requests to departments, and staff access follows their authorized responsibilities.
- SLA timing uses elapsed hours. Business calendars, holidays, waiting-time pauses, priority changes, and reopening need explicit policy definitions before wider use.
- A student reply resumes work only when that student owns the ticket and its current state is `WAITING_FOR_STUDENT`. Ageing reports concern unresolved tickets.
- Seeded accounts and tickets are demonstration data. Production credentials, real student data, and database secrets are kept outside the submission artifacts.

Engineering decisions and tradeoffs

Separate Next.js and Express applications keep UI and service responsibilities clear but require two runtime configurations. PostgreSQL suits the linked ticket data and reporting; Prisma adds typed access and migrations while still requiring careful query and transaction design. A transition map makes workflow rules explicit, and timestamp-based SLA indicators remain current when evaluated. The prototype prioritizes the demonstrated lifecycle over a production operations platform.

Checks to complete before submission

- **Escalation and reopening:** document and demonstrate the actual trigger, responsible role, status behavior, and history entry. If unfinished, identify the feature as a limitation.
- **Concurrent claiming:** send two overlapping claims for the same unassigned ticket. Confirm one succeeds, the other conflicts, and ownership and activity records remain consistent. Verify the database guard. [1]
- **Access and deadlines:** retain direct API checks for another student's ticket, forbidden admin actions, internal-note leakage, and SLA and ageing boundaries. Run these separately if the current tests do not cover them.
- **Fresh setup and demo access:** verify migrations, seed data, role logins, configuration examples, and startup commands from a clean checkout. Check that demo switching cannot elevate API permissions.

Submission package

Include the repository or source archive, a working prototype or reproducible local setup, this report, and a README with environment variables, database setup, run commands, and demo access for all three roles. Retain test and build output and the lifecycle script. Record the submitted commit so reviewers can match the evidence to the code.

Replace the highlighted access fields on page 1. The assignment brief lists **tech_interview@edumerge.com** as the submission address and **9:00 AM on 25 September 2026** as the deadline.

References

Assignment source: Edumerge Solutions, Pre-Drive Product Engineering Assignments, Assignment 4 and submission requirements, pages 4 and 6.

[1] PostgreSQL, Transaction Isolation. <https://www.postgresql.org/docs/current/transaction-iso.html>

[2] Next.js, Missing Suspense boundary with useSearchParams. <https://nextjs.org/docs/messages/missing-suspense-with-csr-bailout>

[3] PostgreSQL, Trust Authentication. <https://www.postgresql.org/docs/current/auth-trust.html>